AI-Powered Document Intelligence System

Dataset guide and step-by-step plan to finish and submit by tomorrow

Prepared for Ahmed Faizul Haque Dhrubo · 10 November 2026 · Target submission: 11 November 2026

How to use this guide. Section 1 is the strategy. Sections 2–4 cover the three datasets (links, formats, verified samples, download code, field mapping). Sections 5–7 cover the files you generate, the unified ground-truth format and the metrics. Section 8 is the hour-by-hour plan. Sections 9–12 are reference material (stack, prompts, rules, templates) and the final submission checklist. Every dataset link is in Section 13.

1. Strategy in one page

Build the **ML core first** as a plain Python package (ingestion, OCR, extraction, validation) and evaluate it from the command line today. Tomorrow, wrap it in FastAPI + Postgres + a Redis queue, build a small Next.js UI, write tests, and package the deliverables. A narrow system that works end to end and reports honest, measured numbers beats a wide one that half works.

Task in brief	What “done” looks like for this submission
1 Document processing	PDF (digital + scanned), JPG/PNG/TIFF, XLSX/XLS/CSV all parsed; corrupted files fail cleanly with an error message
2 AI extraction	9 fields + line items as JSON; null when absent; grounding check flags values not found in the source text
3 Validation	Missing fields, bad numbers, line-item/subtotal/tax/total math, duplicates → needs_review
4 Backend API	Upload, status, results, search, PATCH correction, reprocess; background worker (RQ)
5 Database + search	Postgres tables + full-text search + pgvector semantic search
6 Frontend	Upload + progress, detail/correct, search (3 pages)
7 Tests + evaluation	pytest results file + eval report with measured accuracy, time, success and failure rates

2. Dataset 1 — Katana ML “Sparrow” invoices (main labeled set)

Item	Details
Link	https://huggingface.co/datasets/katanaml-org/invoices-donut-data-v1
License	MIT
Size	501 rows: train 425 · validation 50 · test 26 (the card says 500 documents)
Format	Each row: `image` (invoice image, ~2480 px wide) + `ground_truth` (JSON string)
Type	Synthetic invoices rendered as images → exercises your **image → OCR → LLM** path
Use	validation + test splits = 76 images. Do not tune your prompt on the test split.

Verified ground-truth sample (test split, row 0)

```
{"gt_parse": {
  "header": {"invoice_no": "97159829", "invoice_date": "09/18/2015",
             "seller": "Bradley-Andrade 9879 Elizabeth Common Lake Jonathan, RI 12335",
             "client": "Castro PLC Unit 9678 Box 9664 DPO AP 69387",
             "seller_tax_id": "985-73-8194", "client_tax_id": "994-72-1270", "iban": "GB81..."},
  "items": [{"item_desc": "12\" Marble Lapis Inlay Chess Table Top ...", "item_qty": "2,00",
             "item_net_price": "444,60", "item_net_worth": "889,20",
             "item_vat": "10%", "item_gross_worth": "978,12"}],
  "summary": {"total_net_worth": "$ 889,20", "total_vat": "$ 88,92", "total_gross_worth": "$ 978,12"}}}
```

Mapping to your 9 fields

Your field	Katana key	Normalization
invoice_number	header.invoice_no	string, strip spaces
invoice_date	header.invoice_date	MM/DD/YYYY → ISO YYYY-MM-DD
supplier_name	header.seller	name + address are merged; compare with fuzzy match (see §7)
customer_name	header.client	same as supplier
subtotal	summary.total_net_worth	“$ 889,20” → 889.20 (comma is the decimal separator)
tax	summary.total_vat	same
total	summary.total_gross_worth	same
currency	symbol in summary	“$” → USD
line_items[]	items[]	desc, qty, unit_price=item_net_price, amount=item_net_worth, vat %, gross

Quirks to handle in your loader: amounts use a comma as the decimal separator (“889,20”); some rows omit `invoice_no`, some have no `header` wrapper, and at least one wraps everything under a `"None"` key. Write a tolerant normalizer and skip/flag rows that cannot be parsed. Bonus: the math is consistent (889.20 + 88.92 = 978.12), so these invoices are good positive cases for your validator.

Download

```
# pip install datasets pillow
from datasets import load_dataset
import os, json
ds = load_dataset("katanaml-org/invoices-donut-data-v1")
os.makedirs("data/raw/katana", exist_ok=True)
for split in ["validation", "test"]:
    for i, row in enumerate(ds[split]):
        stem = f"data/raw/katana/{split}_{i:03d}"
        row["image"].convert("RGB").save(stem + ".png")
        with open(stem + ".json", "w") as f:
            f.write(row["ground_truth"])
```

3. Dataset 2 — Northwind invoices (digital PDFs + PDF↔Excel)

Item	Details
Link	https://huggingface.co/datasets/AyoubChLin/northwind_invocies (the typo “invocies” is the real repo name)
License	Apache-2.0
Size	831 one-page PDFs named `invoice_10248.pdf` … plus one stray `invoice.pdf` (skip it). No CSV is in the repo.
Type	Generated from the Northwind sample database → exercises your **digital PDF** path
Use	50 PDFs for accuracy; all 831 merged into one long PDF for the speed/splitting test

Verified text of invoice_10253.pdf

```
                                INVOICE
Invoice Number: 10253
Customer Name: Hanari Carnes
Salesperson:        Janet Leverling
Invoice Date:       2016-07-10

Product Name                            Unit Price   Quantity   Total Price
Gorgonzola Telino                       $10.00       20         $200.00
Chartreuse verte                        $14.40       42         $604.80
Maxilaku                                $16.00       40         $640.00
                                                     Total:     $1444.80
```

What this dataset tests

- Present: invoice number, customer name, date (already ISO), line items, total, “$”.
- **Absent: supplier name, subtotal, tax.** The ground truth for these is `null`. This directly tests the requirement “handle missing information without inventing values”. If your LLM writes “Northwind Traders” as the supplier or invents a tax of 0.00, count it as a hallucination.
- Currency: only “$” is printed. Accept USD, and say in the report that it was inferred from the symbol.

Ground truth: two options

- **Option A (fastest, recommended today):** parse the PDF text layer with regex (the text is clean and machine-generated), then hand-check 20 files. Your extraction pipeline must not use this parser.
- **Option B (for the PDF↔Excel cross-check):** the Northwind database exported to XLSX/CSV. A SQLite copy is at https://github.com/jpwhite3/northwind-SQLite3 (file `dist/northwind.db` is stored with Git LFS, so download it through the GitHub web page or with `git lfs pull`). Join Orders + “Order Details” + Customers, filter OrderID 10248–11077, and write `ledger.xlsx`. **Check 3–4 orders against the PDFs first** (dates, prices); the PDFs show 2016 dates, while the classic database uses 1996–1998 dates. The database also has a Discount column; if a PDF total ignores a discount, that is a real mismatch your cross-check should report.

```
import re, fitz  # pip install pymupdf
def northwind_gt(path):
    t = fitz.open(path)[0].get_text()
    g = lambda p: (m.group(1).strip() if (m := re.search(p, t)) else None)
    items = [dict(description=d.strip(), unit_price=float(u), quantity=int(q), amount=float(a))
             for d, u, q, a in re.findall(r"^(.+?)\s+\$([\d.]+)\s+(\d+)\s+\$([\d.]+)\s*$", t, re.M)]
    return {"invoice_number": g(r"Invoice Number:\s*(\S+)"), "invoice_date": g(r"Invoice Date:\s*(\S+)"),
            "supplier_name": None, "customer_name": g(r"Customer Name:\s*(.+)"),
            "subtotal": None, "tax": None, "total": float(g(r"Total:\s*\$([\d.,]+)").replace(",", "")),
            "currency": "USD", "line_items": items}
```

Check the regex on the layout of your own download: PyMuPDF may order the table text differently from `pdftotext -layout`. If the line-item regex misses rows, use `page.get_text("words")` or pdfplumber tables instead.

Download + extra test files

```
from huggingface_hub import snapshot_download   # pip install huggingface_hub
snapshot_download("AyoubChLin/northwind_invocies", repo_type="dataset",
                  local_dir="data/raw/northwind", allow_patterns=["invoice_*.pdf"])

# One ~830-page PDF for the speed + invoice-splitting test
import fitz, glob
out = fitz.open()
for p in sorted(glob.glob("data/raw/northwind/invoice_1*.pdf")):
    out.insert_pdf(fitz.open(p))
out.save("data/eval/long/northwind_merged.pdf")

# "Scanned" variant: render pages to images (add blur/noise/rotation later)
pix = fitz.open("data/raw/northwind/invoice_10253.pdf")[0].get_pixmap(dpi=200)
pix.save("data/eval/scanned/invoice_10253.png")
```

4. Dataset 3 — SROIE scanned receipts (real, poor-quality scans)

Item	Details
Link	https://huggingface.co/datasets/Voxel51/scanned_receipts
Original	ICDAR 2019 SROIE challenge: https://rrc.cvc.uab.es/?ch=13 · paper arXiv 2103.10213
License	CC-BY-4.0
Size	712 JPG receipts in `data/`; labels for all of them in `samples.json` (~23 MB)
Labels	company, date, address, total (plus OCR word boxes you don't need)
Use	30 receipts, scoring supplier (= company), date and total only

Verified sample from samples.json

```
{"filepath": "data/X51005442376.jpg",
 "company": "UNIHAKKA INTERNATIONAL SDN BHD", "date": "22 MAR 2018",
 "address": "12, JALAN TAMPOI 7/4, KAWASAN PERINDUSTRIAN TAMPOI, 81200 JOHOR BAHRU, JOHOR",
 "total": "$7.60"}
```

Download (no FiftyOne needed)

```
from huggingface_hub import snapshot_download
import json, random
snapshot_download("Voxel51/scanned_receipts", repo_type="dataset",
                  local_dir="data/raw/sroie", allow_patterns=["data/*.jpg", "samples.json"])
samples = json.load(open("data/raw/sroie/samples.json"))["samples"]
random.seed(42)
for s in random.sample(samples, 30):
    gt = {"supplier_name": s["company"], "invoice_date": s["date"], "total": s["total"]}
    # copy data/raw/sroie/<s["filepath"]> into data/eval/sroie/ and save gt next to it
```

Mapping: company → supplier_name; date → invoice_date (formats vary: “22 MAR 2018”, “25/12/2018”; parse with `dateutil.parser.parse(dayfirst=True)`); total → total (strip the symbol). These are Malaysian receipts, not B2B invoices, and have no invoice number, customer or tax labels. Report SROIE separately as “real scanned documents, partial labels”.

5. Files you generate yourself

No public dataset covers spreadsheets, corrupted files or duplicates, so write `eval/prepare_dataset.py` to create them:

Set	How to make it	Count	Expected result
XLSX invoices	Northwind ledger (Option B) or Faker; one sheet per invoice + one multi-sheet workbook	10	Parsed, all sheets read
CSV invoices	Same data; include one cp1252-encoded file and one semicolon-delimited file	10	Parsed, encoding/delimiter detected
Degraded scans	Northwind pages rendered at 150–200 DPI + Gaussian blur, ±3° rotation, JPEG quality 40	10	OCR path; lower accuracy is fine
Corrupted / edge	Truncated PDF (first 40% of bytes), 0-byte file, JPG renamed .pdf, password PDF, random bytes .xlsx	5–10	status = failed, clear error, no crash
Duplicates	Same invoice uploaded twice; same invoice as PDF and PNG	5	flagged duplicate
Planted errors	Copy Katana/Northwind invoices and change total or one line amount (re-render for images)	10	flagged math error
Long PDF	Merged Northwind file (§3)	1 (~830 pages)	Split into ~830 invoices; report time

6. One ground-truth format for everything

Convert every source into the same JSON so one evaluator scores all of them. Put each file next to its document and list everything in `data/eval/manifest.csv`.

```
// data/eval/gt/<doc_id>.json
{
  "doc_id": "katana_test_003", "source": "katana", "file": "data/eval/katana/test_003.png",
  "file_type": "image", "expect_failure": false, "planted_error": null,
  "labeled_fields": ["invoice_number","invoice_date","supplier_name","customer_name",
                     "subtotal","tax","total","currency","line_items"],
  "invoice": {
    "invoice_number": "97159829", "invoice_date": "2015-09-18",
    "supplier_name": "Bradley-Andrade", "customer_name": "Castro PLC",
    "subtotal": 889.20, "tax": 88.92, "total": 978.12, "currency": "USD",
    "line_items": [{"description": "...", "quantity": 2, "unit_price": 444.60, "amount": 889.20}]
  }
}
```

`manifest.csv` columns: `doc_id, source, file, file_type, expect_failure, planted_error, labeled_fields`. `labeled_fields` matters: SROIE only labels 3 fields, so only those are scored. Fields labeled `null` (Northwind supplier/subtotal/tax) are scored too: the correct prediction is null.

Final evaluation set

Source	Docs	Path tested	Fields scored
Katana (validation + test)	76	Image → OCR → LLM	All 9 + line items
Northwind PDFs	50	Digital PDF → LLM	invoice no., date, customer, total, line items; supplier/subtotal/tax must be null
Northwind merged PDF	1 (~830 pages)	Long PDF, splitting, speed	Invoice count found, time/page
SROIE	30	Real poor scans	supplier, date, total
XLSX + CSV	20	Spreadsheets + PDF↔Excel	All fields present in the sheet
Degraded scans	10	OCR robustness	Same as Northwind
Corrupted / edge	5–10	Failure handling	Fails cleanly (yes/no)
Duplicates + planted errors	15	Validation	Flag raised (yes/no)

7. Metrics: exactly what to measure

Metric	Definition
Field accuracy	Per field: correct ÷ labeled. Text fields: case/space-insensitive exact match, or fuzzy token-set ratio ≥ 0.85 for names (rapidfuzz). Supplier/customer in Katana: also accept a match against the name part only. Dates: equal after ISO normalization. Amounts: |pred − gt| ≤ 0.01. Null fields: correct only if pred is null.
Hallucination rate	Fields where gt is null but the system returned a value ÷ all null-gt fields
Line items	Count match rate; plus item-level precision/recall, matching rows by description (fuzzy ≥ 0.8) and amount
Document accuracy	Share of documents where every scored field is correct
Processing time	p50 and p95 seconds per document and per page, reported separately for digital PDF, OCR and spreadsheets; plus total time for the merged long PDF
Success rate	Documents processed without error ÷ documents expected to succeed
Failure rate	Unexpected failures ÷ all documents. Expected failures (corrupted files) that fail cleanly count as handled, not failures
Validation detection	Planted errors and duplicates flagged ÷ planted; false flags on clean docs ÷ clean docs

Only report numbers produced by your evaluator script. Save the raw per-document results (`eval/results.jsonl`) in the repo so reviewers can reproduce every figure.

8. Step-by-step plan until submission

Times are estimates for focused work. If you fall behind, use the cut list at the end of this section rather than skipping tests or the evaluation.

TODAY — ML core and first measured results

Step	Time	Do this	Done when
1 Setup	0:45	Create the repo and folders (§9). Python 3.11 venv. Install: pymupdf, pdfplumber, pytesseract, opencv-python-headless, pillow, pandas, openpyxl, xlrd, charset-normalizer, python-magic, pydantic, rapidfuzz, python-dateutil, your LLM SDK, pytest. Install Tesseract. Put the API key in .env (never commit it).	`tesseract --version` works; first commit pushed
2 Data	1:15	Download the 3 datasets (§2–4). Write `eval/prepare_dataset.py`: copy the chosen samples, convert labels to the unified format (§6), create the generated files (§5), write manifest.csv.	manifest.csv lists ~210 docs; spot-check 5 GT files by eye
3 Ingestion	2:30	`ingest/router.py` (detect type by content), `pdf.py` (text per page; OCR if <50 chars), `image.py` (grayscale, deskew, threshold, upscale small images; multi-frame TIFF), `sheet.py` (all sheets, encoding + delimiter detection). Wrap everything in try/except → structured error. CLI that dumps text for a folder.	CLI runs on every sample type; corrupted files return errors, no crash
4 Extraction	2:30	Pydantic schema (all Optional). Prompt (§10) with JSON output, temperature 0. `normalize.py` (dates, Decimal amounts incl. “889,20”, currency). `grounding.py`: each value must appear in the source text (fuzzy), otherwise mark uncertain. `segment.py`: split long PDFs at pages that start a new invoice.	10 Katana + 10 Northwind docs give sensible JSON; Northwind supplier/tax come back null
5 Validation	1:00	`validate.py` with the rules in §10, duplicate check (same supplier + invoice no., same file hash), simple cross-verify (match by invoice no., list field mismatches).	Planted errors are flagged on 3 hand-picked files
6 First eval	1:00	`eval/evaluate.py`: run the pipeline over manifest.csv, write results.jsonl and a metrics table. Look at the worst 10 failures and fix the top 1–2 causes.	You have real numbers; commit them

TOMORROW — wrap, show, prove, submit

Step	Time	Do this	Done when
7 API + DB	2:30	Docker Compose: Postgres (pgvector image) + Redis + API + worker. Tables from §10. FastAPI endpoints (§10). Upload saves the file, inserts a row, enqueues an RQ job that calls `process_file()` and stores results + events. PATCH re-runs validation and writes a corrections row.	curl upload → poll status → get validated JSON
8 Search	0:45	Postgres tsvector + GIN index for keywords and filters; sentence-transformers (e.g. all-MiniLM-L6-v2, 384-dim) embeddings in pgvector; `/search?mode=keyword|semantic|hybrid`.	"marble chess table" finds the right invoice
9 Frontend	2:30	Next.js: (1) drag-drop multi-upload + progress table polling status; (2) document detail: fields, confidence, validation issues, editable values → PATCH; (3) search page with filters.	Full flow works in the browser
10 Tests	1:15	pytest: one test per file type, corrupted + empty files, each validation rule, extraction with the LLM mocked, API tests with TestClient, one reprocess-after-failure test. `pytest --junitxml=test-results.xml --cov`.	All green; results file committed
11 Final eval	0:45	Re-run evaluate.py (through the API if time allows) and write `docs/eval_report.md` from the template in §11.	Report with tables + failure analysis
12 Docs	1:15	README (setup, run, design decisions, security, limitations), architecture diagram (Mermaid), DB schema, AI disclosure + prompts file.	A stranger could run it from the README
13 Video	0:45	5–7 minute screen recording following the script in §11.	Video link in README
14 Submit	0:30	Go through the checklist in §12, tag a release, push, send the link.	Submitted

Cut list (if you fall behind, drop in this order)

- Semantic search → keep keyword search only and say so in the README.
- Degraded-scan set and FATURA/CORD extras.
- Line-item precision/recall → report count match + sum check only.
- Running the evaluation through the API → run it on the Python package and say so.
- Never drop: the tests file, the measured eval report, the README setup steps, the video.

9. Recommended stack and repo layout

Layer	Choice	One-line justification for the README
API	FastAPI	Python end to end; Pydantic schemas; free OpenAPI docs
Queue	Redis + RQ	Simplest reliable background jobs in Python; fills the Redis/BullMQ role
Database	PostgreSQL 16 + pgvector + pg_trgm	One store for relational data, full-text, fuzzy matching and vectors
OCR	Tesseract 5 (PaddleOCR optional on GPU)	Easy Docker install; swap in PaddleOCR on bad scans
LLM	API with JSON/structured output (or local Qwen via Ollama)	Pluggable provider class; temperature 0
Embeddings	sentence-transformers, 384-dim	Runs locally, fast
Frontend	Next.js + Tailwind	Required by the task
Deploy	Docker Compose	One command for the reviewer

```
docintel/
  docintel/            # ML core (built today)
    ingest/{router,pdf,image,sheet}.py
    segment.py  pipeline.py  validate.py  crossverify.py  cli.py
    extract/{schema,prompts,llm,grounding,normalize}.py
  api/                 # FastAPI app, routes, db models, RQ worker (tomorrow)
  web/                 # Next.js frontend
  eval/{prepare_dataset,evaluate}.py   results.jsonl
  data/eval/           # small samples committed; full raw data git-ignored
  tests/               # pytest + fixtures
  docs/{architecture.md, db_schema.md, eval_report.md, ai_disclosure.md, prompts.md}
  docker-compose.yml  .env.example  README.md
```

10. Core implementation reference

Extraction prompt (starting point)

```
SYSTEM: You extract invoice data from document text. Return ONLY JSON matching the schema.
Rules:
- Use only information present in the text. If a field is not present, return null.
  Never guess, compute, or fill in a value that is not written in the document.
- Copy invoice_number exactly as written. Dates: return them as written; do not reformat.
- Amounts: return the number as written including its decimal separator.
- For each field return {"value": ..., "confidence": 0-1, "evidence": "<short exact quote>"}.
- line_items: one object per table row with description, quantity, unit_price, amount.
USER: <document text, with page markers like [PAGE 3]>
```

Normalize dates and amounts in Python after the LLM call, not inside the prompt. Your grounding check then verifies that each `evidence` quote really appears in the source text.

Validation rules (tolerance: 0.01 absolute or 0.5%, whichever is larger)

- Required field missing: invoice_number, invoice_date, supplier_name, total.
- Unparseable or negative amount; date invalid or in the future.
- Line item: quantity × unit_price ≠ amount.
- Sum of line amounts ≠ subtotal (or ≠ total when there is no subtotal and no tax).
- subtotal + tax ≠ total; implied tax rate outside 0–30%.
- Duplicate: same supplier + invoice_number, or same file SHA-256, or same supplier + date + total.
- Low confidence (< 0.6) or a grounding failure on any field → needs_review.
- Cross-verify (PDF↔PDF, PDF↔Excel): match by invoice_number; report each field that differs.

API endpoints

```
POST  /documents                 multi-file upload → [{document_id, status:"queued"}]
GET   /documents/{id}             status, progress (pages done / total), events
GET   /documents/{id}/results     invoices + line items + validation issues
GET   /invoices?supplier=&date_from=&date_to=&min_total=&status=&q=
GET   /search?q=...&mode=keyword|semantic|hybrid
PATCH /invoices/{id}              {field: value} → save, re-validate, write corrections row
POST  /documents/{id}/reprocess   re-queue a failed or edited document
POST  /cross-verify               {document_ids:[a,b]} → mismatch report
GET   /health
```

Database tables

```
documents(id, filename, mime, sha256 UNIQUE, status, page_count, error, created_at)
pages(id, document_id, page_no, text, method, ocr_confidence)
invoices(id, document_id, page_from, page_to, invoice_number, invoice_date, supplier_name,
         customer_name, subtotal NUMERIC, tax NUMERIC, total NUMERIC, currency,
         field_meta JSONB, status, created_at)
line_items(id, invoice_id, description, quantity, unit_price, amount)
validation_issues(id, invoice_id, rule, severity, message, resolved)
processing_events(id, document_id, stage, status, duration_ms, message, created_at)
corrections(id, invoice_id, field, old_value, new_value, corrected_at)
chunks(id, document_id, page_no, text, tsv TSVECTOR, embedding VECTOR(384))
```

Security notes for the README

- File type checked by content (python-magic), size limit per file, randomized storage names.
- Spreadsheets read as data only (no macros); PDFs opened read-only; per-job timeouts.
- API key header, CORS restricted to the frontend origin, secrets only in .env.
- Parameterized SQL through the ORM; uploads stored outside any served directory.

11. Templates

Evaluation report (docs/eval_report.md)

- **Setup:** machine (e.g. RTX 3090, 62 GB RAM), LLM and OCR versions, date, number of docs per source.
- **Datasets:** the table from §6 with links and licenses; state which sets are synthetic and which are real.
- **Results:** field accuracy per source; hallucination rate; line-item metrics; time p50/p95 per type; merged-PDF time; success and failure rates; validation detection.
- **Failure analysis:** 5–10 concrete examples (what went wrong and why: OCR error, segmentation, prompt, normalization).
- **Limitations and next steps:** e.g. DocILE for real B2B invoices, PaddleOCR, fine-tuned layout models.

AI disclosure (docs/ai_disclosure.md)

- Which AI tools you used and for what (planning, code generation, debugging, documentation). This guide was prepared with Claude, so include that.
- Which parts you wrote or reviewed yourself, and how you verified AI-generated code (tests, manual checks).
- The exact runtime prompts used by the system (link docs/prompts.md).

Video script (5–7 minutes)

Time	Show
0:00–0:45	Problem and architecture diagram
0:45–2:30	Upload a mix: digital PDF, scanned image, XLSX, CSV, a corrupted file. Show progress and the failed file's error.
2:30–4:00	Open an invoice: extracted fields, null fields, a validation error; correct a value and show re-validation.
4:00–5:00	Search: keyword filter and a semantic query. Show reprocessing of a failed job.
5:00–6:30	Terminal: pytest passing, evaluation numbers, the long-PDF timing. Close with limitations.

12. Submission checklist

#	Deliverable	Where	✓
1	Git repository with source code	GitHub link	
2	README and setup instructions	README.md	
3	Architecture diagram and database schema	docs/architecture.md, docs/db_schema.md	
4	Working backend API and frontend	docker compose up; /docs; web/	
5	PDF, image, Excel and CSV pipelines	docintel/ingest/	
6	Automated tests and test results	tests/, test-results.xml, coverage	
7	Evaluation report with measured metrics	docs/eval_report.md, eval/results.jsonl	
8	Recorded walkthrough	Video link in README	
9	AI prompts and AI-assistance disclosure	docs/prompts.md, docs/ai_disclosure.md	

Last checks before sending: fresh clone → follow README → app starts; no API keys in git history; dataset licenses credited; large raw data git-ignored with download instructions instead.

13. All dataset links

Dataset	Link	License
Katana ML Sparrow invoices	https://huggingface.co/datasets/katanaml-org/invoices-donut-data-v1	MIT
Northwind invoices (PDF)	https://huggingface.co/datasets/AyoubChLin/northwind_invocies	Apache-2.0
Northwind database (SQLite, for the ledger)	https://github.com/jpwhite3/northwind-SQLite3	see repo
SROIE (Voxel51 mirror)	https://huggingface.co/datasets/Voxel51/scanned_receipts	CC-BY-4.0
SROIE original challenge	https://rrc.cvc.uab.es/?ch=13	see site
SROIE paper	https://arxiv.org/abs/2103.10213	—
Optional: CORD v2 receipts	https://huggingface.co/datasets/naver-clova-ix/cord-v2	CC-BY-4.0
Optional: FATURA (50 layouts)	https://zenodo.org/records/10371464	see record
Future work: DocILE (needs token)	https://docile.rossum.ai/	see site

Dataset facts in this guide (splits, file names, label formats, sample records) were checked against the Hugging Face pages and files on 10 November 2026. Recheck licenses before redistributing any data.